CPAD ESE Mini Project

SeatSure

Book library seats by time slot, with QR check-in and automatic release of no-show bookings

Student	Chaitanya Shinde (231070066), Final Year B.Tech Computer Engineering
Course	Cross Platform App Development Lab (R5CO4004L), Semester VII, 2026–27
Framework	Flutter (the one CPAD framework used), with a Node.js + Express backend and PostgreSQL
Live app	seatsure-9hn0.onrender.com
Source code	github.com/Chaitanya-666/CpadProjectSeatSure

Abstract

College reading halls fill up with bags on empty chairs: a seat is “taken” for hours while nobody uses it. SeatSure is a cross-platform app that lets a student book a library seat for a time slot, proves they actually came by having them scan a QR code stuck on the desk, and releases the seat automatically if they do not check in within 15 minutes. A librarian sees live occupancy, no-show rates and the busiest hours, and manages rooms, seats and the printed QR codes.

The app is written once in Flutter and runs on Android, in a web browser and as a Linux desktop app. All three talk to one REST API (Node.js + Express) backed by PostgreSQL. The rule that matters most, “no seat may be booked twice for overlapping times”, is enforced by the database itself with an exclusion constraint, so it holds even when two phones press “Book” at the same instant. The system is deployed (Render + Neon), has 36 automated tests that run on every push, and was tested on an Android emulator and on the live deployment.

Contents

Introduction 4
- Problem statement 4
- Proposed solution 4
- Objectives 4
- Scope 4

Requirements 4
- Users 4
- Functional requirements 5
- Non-functional requirements 5

Technology stack 6
System architecture 7
Database design 8
Core logic 10
- Booking a seat (transaction) 10
- Booking status 11
- Lazy no-show release 11
- QR check-in 11
- Time rules 11

API reference 12
Security 12
The Flutter app 13
Screens 14
- Student (Android emulator, light theme) 14
- Librarian (admin) 17
- Dark theme and the web build 18

Testing 20
- Automated tests 20
- Continuous integration 21
- Manual test rounds 21

Deployment 22

Limitations and future scope 22
Conclusion 23
Appendix A: Running the project 23
Appendix B: Where is X 23

Introduction

Problem statement

A college library has a fixed number of reading seats and many more students who want them, especially before exams. Three things go wrong today:

1. **Seat hogging.** A student leaves a bag on a chair and goes away for hours. The seat looks taken but nobody uses it.
2. **Wasted trips.** A student walks to the library without knowing whether any seat is free.
3. **No data.** The librarian cannot tell which rooms are busiest, at what hours, or how often people book and never come.

Proposed solution

SeatSure turns a seat into a time-slot booking that must be **claimed in person**:

- Students see each room's free seats live and book one for a slot (30 minutes to 4 hours, up to 7 days ahead).
- Every desk has a QR code. Scanning it within the check-in window proves the student is at that exact seat.
- If nobody checks in within 15 minutes of the start, the booking becomes a **no-show** and the seat is released for others.
- The librarian gets a dashboard built from the booking data.

Objectives

- Build one Flutter codebase that runs on Android, the web and Linux desktop.
- Integrate a real backend (REST API) and a relational database, as the ESE guidelines require.
- Make double booking impossible, even under simultaneous requests.
- Release no-show seats reliably, even on a free server that sleeps when idle.
- Use a device capability (the camera) for QR check-in.
- Test it automatically and on real devices, and deploy it so it works from anywhere.

Scope

In scope	**Out of scope (future work, Section 12)**
Student sign-up and login; admin (librarian) role	College single sign-on, email verification, password reset
Rooms with opening hours; seats with labels, power sockets, QR codes	Group study rooms booked as a whole; seat maps drawn to the room's real layout
Booking, cancel, QR check-in, leave early, automatic no-show release	Push notifications when the server releases a seat; waitlists
Admin dashboard, day list, room and seat management, QR sheet	Penalties for repeated no-shows
Android, web and Linux desktop builds	iOS and macOS builds (need a Mac with Xcode)

Requirements

Users

User	**What they need**

Student	See free seats, book one, check in at the desk, leave early or cancel, see their history.
Librarian (admin)	Watch live occupancy, see who booked what, add rooms and seats, print and replace QR codes, see no-show rates and busy hours.

Functional requirements

ID	Requirement
FR-1	A visitor can create a student account (name, email, password ≥ 8 characters) and log in. Accounts can never make themselves admin.
FR-2	A student sees all open rooms with opening hours and the number of seats free right now.
FR-3	For a chosen day, start time and length, a student sees every seat of a room as free, taken or theirs, refreshed every 15 seconds.
FR-4	A student books a free seat. Start and end are on a 15-minute grid, the length is 30 minutes to 4 hours, the start is not in the past and at most 7 days ahead, and the slot is inside the room's opening hours.
FR-5	Two bookings of the same seat may never overlap in time. One student may never hold two seats at the same time, and may hold at most 3 upcoming bookings.
FR-6	A student checks in by scanning the seat's QR code (or typing the code printed under it) from 10 minutes before the start until 15 minutes after it. A wrong seat, too early, too late or no booking each get a clear message.
FR-7	A booking not checked in by start + 15 minutes becomes a no-show and the seat becomes free.
FR-8	A student can cancel a booking before checking in, and can "leave early" after checking in, which frees the seat at once.
FR-9	A student sees upcoming bookings with a live countdown to the check-in deadline, and past bookings with their outcome.
FR-10	An admin sees live counts (in use, waiting check-in, free), per-room bookings, attendance, no-shows, no-show rate and seat-hours used over the last 7 days, and the busiest start hours.
FR-11	An admin sees every booking of a chosen day with the student's name and email.
FR-12	An admin adds rooms (name, floor, hours), opens or closes a room, adds seats in bulk, disables a seat, shows each seat's QR code, and issues a new QR code that makes the old one stop working.

Non-functional requirements

ID	Requirement
NFR-1 Correctness	Booking rules hold under concurrent requests (enforced in the database, proven by tests).
NFR-2 Security	Passwords stored only as slow salted hashes; tokens expire; admin routes check the role on the server; login is rate limited; no secret is ever committed to the public repository.
NFR-3 Portability	One codebase for Android, web and Linux desktop.
NFR-4 Usability	Material 3 design with light and dark themes; phone layout with bottom tabs, wide layout with a side rail; readable error messages.

NFR-5 Availability	Works on free hosting: tolerates the server sleeping, and the database URL and secrets come from the environment.
NFR-6 Maintainability	Automated tests and CI on every push; schema changes as numbered SQL migrations.

Technology stack

Layer	Choice (version)	Why
App	Flutter 3.47.7, Dart 3.13.5	The course's main framework (Exp 1–5). One codebase builds a native Android app, a web app and a Linux desktop app. The ESE rule "only ONE of Flutter / React Native / Xamarin" is met: the whole client is Flutter.
App state	provider 6.1.5	`ChangeNotifier` + `Provider`, the same pattern used in Exp 1–3, kept deliberately simple.
App packages	http 1.6.0, shared_preferences 2.5.6, mobile_scanner 7.4.3, qr_flutter 4.1.0, intl 0.20.3	HTTP calls; saving the login; camera QR scanning (ML Kit on Android); drawing QR codes; date formats.
API	Node.js 24 (26 locally), Express 5.3.0	Small, well-known REST framework. Express 5 forwards errors from `async` handlers to the error handler automatically.
Validation	zod 4.6.7	Every request body and query is checked against a schema before any SQL runs.
Auth	jsonwebtoken 9.0.3, Node `crypto.scrypt`	Stateless Bearer tokens; scrypt is a slow, memory-hard password hash built into Node (no native add-on).
Security middleware	helmet 8.3.0, cors 2.8.6, express-rate-limit 8.7.2	Safe HTTP headers; cross-origin calls from the app; slows password guessing.
Database	PostgreSQL 18.6, driver pg 8.23.1	Relational data (rooms → seats → bookings ← users). Range types and exclusion constraints make "no overlap" a database rule. Hand-written SQL, no ORM, so every query is visible.
Hosting	Render (Docker, free), Neon (Postgres, free), both in Singapore	Free tiers with HTTPS. Neon's free database does not expire (Render's free database does, after about 30 days).
CI	GitHub Actions	API tests against a real Postgres, Flutter analyze + tests, and a build-and-boot check of the Docker image on every push.

Why our own API instead of Firebase or Supabase

Firebase or Supabase would have given a database and login with almost no backend code, and many student projects use them. They were not used because (1) the guidelines ask for an **integrated**

backend, and with our own API every rule (who may book, when check-in is allowed, how no-shows are released) is code that can be shown and explained; (2) the data is relational (rooms, seats, bookings, users), and the per-room statistics are single SQL queries with joins and aggregates, which Firestore (a NoSQL store) does not do; (3) the most important rule, no overlapping bookings, is a one-line PostgreSQL constraint. Neon is used only as a hosted PostgreSQL server: the app never talks to it directly.

System architecture

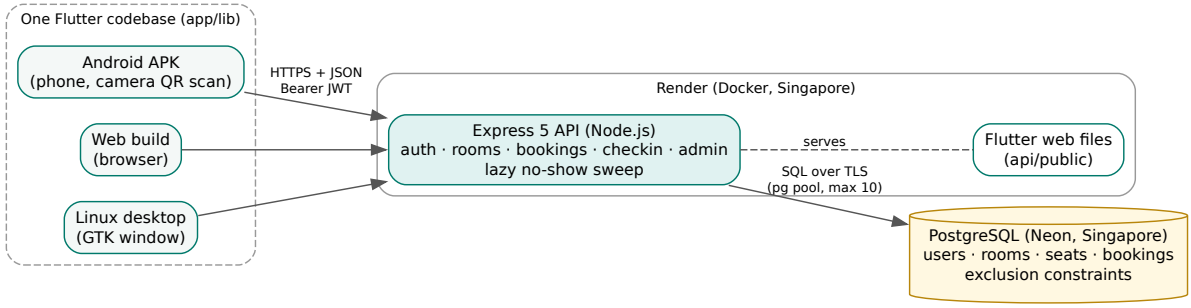


Figure 1: Architecture: three Flutter builds, one API, one database

The system has three tiers:

1. **Client (Flutter).** Screens, input checks, and the camera. It never talks to the database; it only calls the API over HTTPS with JSON bodies, sending `Authorization: Bearer <token>` after login.
2. **API (Express).** Checks the token and the role, validates input, applies the booking rules, runs SQL in transactions, and turns database errors into readable messages. In production it also serves the Flutter web build, so the web app and the API share one address.
3. **Database (PostgreSQL).** Stores everything and is the final judge of the rules that must never break.

One request, end to end: “book seat A3”

1. The student taps A3 and confirms. `BookingsModel.book()` sends `POST /api/bookings` with `{seatId, start, end}` (times converted to UTC ISO strings).
2. `requireAuth` reads the Bearer token, verifies its signature and expiry, and loads the user from the database.
3. `noShowSweep` marks any expired, unclaimed bookings as no-shows (Section 6.3).
4. The route validates the body with zod and the time rules with `validateSlot`, then runs the booking transaction (Section 6.1).
5. PostgreSQL inserts the row, or rejects it with an exclusion-constraint error that the error handler turns into `409 seat_taken`.
6. The app shows “Booked A3. Check in now, before 4:15 PM”, refreshes My bookings, and switches to that tab.

Database design

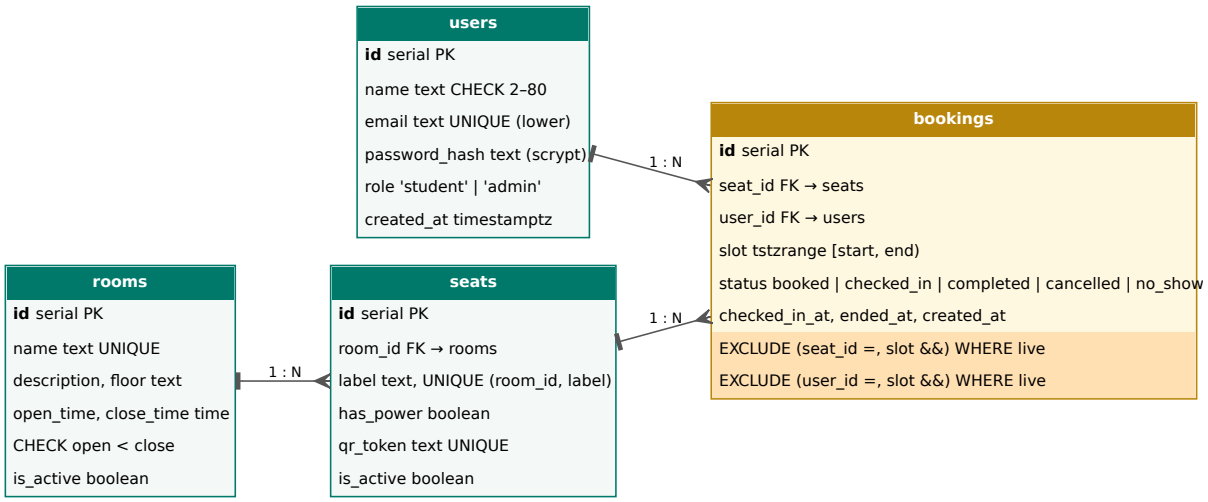


Figure 2: Entity-relationship diagram (`api/migrations/001_init.sql`)

Table	Purpose and key rules
`users`	Students and admins. Email is unique ignoring case (a unique index on `lower(email)`). `role` is checked to be `student` or `admin`. Only the scrypt hash of the password is stored.
`rooms`	A reading hall or zone with opening hours stored as local times; `CHECK (open_time < close_time)`.
`seats`	Belongs to a room; label unique within the room; `qr_token` is a random secret printed as the seat's QR code (unique across all seats).
`bookings`	One row = one seat held for one time range. `slot` is a `tstzrange` (a range of timestamps with time zone). `status` is one of five values (Section 6.2).

The constraint that prevents double booking

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CONSTRAINT no_seat_overlap EXCLUDE USING gist (seat_id WITH =, slot WITH &&)
  WHERE (status IN ('booked', 'checked_in')),
CONSTRAINT no_user_overlap EXCLUDE USING gist (user_id WITH =, slot WITH &&)
  WHERE (status IN ('booked', 'checked_in'))
```

An **exclusion constraint** generalises `UNIQUE`: instead of "no two rows are equal", it says "no two rows have this combination of comparisons all true". Here: no two **live** rows may have the same `seat_id` (=) **and** overlapping slots (&&, the range overlap operator). The `WHERE` clause makes it partial: cancelled, completed and no-show rows drop out, which is exactly how releasing a seat makes it bookable again. `btree_gist` lets one GiST index combine the plain = on an integer with && on a range.

Ranges are half-open, `[start, end)`. So 10:00–11:00 and 11:00–12:00 do not overlap and back-to-back bookings are allowed (tested).

Why the database, and not just an "if" in the API?

An API check like “SELECT overlapping bookings; if none, INSERT” has a race: two requests can both run the SELECT, both see nothing, and both INSERT. The constraint is checked inside PostgreSQL at insert time, under its own locking, so exactly one of the two inserts succeeds. The test “lets exactly one of two simultaneous requests for the same seat win” fires two bookings at once and expects one `201` and one `409`; when the constraint was dropped on purpose, the test failed.

Other constraints and indexes

- `CHECK (NOT isempty(slot) AND NOT lower_inf(slot) AND NOT upper_inf(slot))` and `CHECK (lower_inc(slot) AND NOT upper_inc(slot))`: every slot is a real `[start, end)` range.
- `CHECK ((status IN ('checked_in','completed')) = (checked_in_at IS NOT NULL))`: a check-in time exists exactly when the student checked in.
- Foreign keys with `ON DELETE CASCADE`: deleting a room removes its seats and their bookings.
- `bookings_pending_idx ON bookings (lower(slot)) WHERE status = 'booked'`: a small partial index for the no-show sweep, which only ever looks at unclaimed bookings.
- Schema changes are numbered files in `api/migrations/`, applied once each and recorded in `schema_migrations`; the Docker image applies new ones at start-up.

Core logic

Booking a seat (transaction)

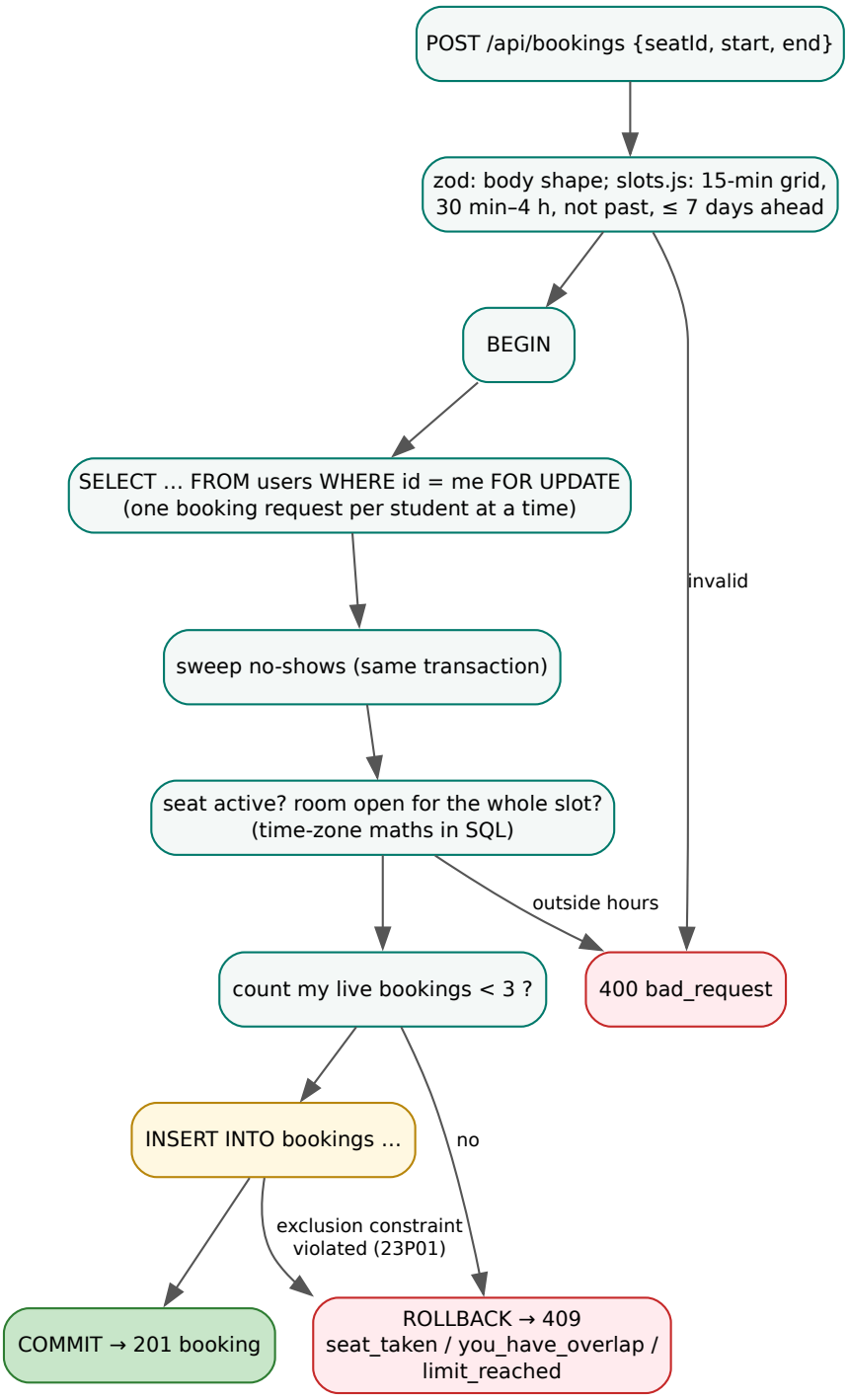


Figure 3: `POST /api/bookings` in `api/src/routes/bookings.js`

The whole booking runs in one transaction (`withTransaction` in `api/src/db.js`), so any error rolls everything back. Two details matter:

- **Row lock on the student** (`SELECT … FOR UPDATE`). The "at most 3 upcoming bookings" rule is a count followed by an insert, which has the same race as above. Locking the student's own `users` row

makes two parallel requests from the same student run one after the other. Removing the lock made the test “enforces the per-student limit even for parallel requests” fail 3 times out of 3.

- **Opening hours in SQL.** Rooms store hours as local time (`APP_TIMEZONE = Asia/Kolkata`). The check converts the slot with `AT TIME ZONE` inside PostgreSQL, so time-zone rules come from the database’s time-zone data and not from hand-written arithmetic.

Booking status

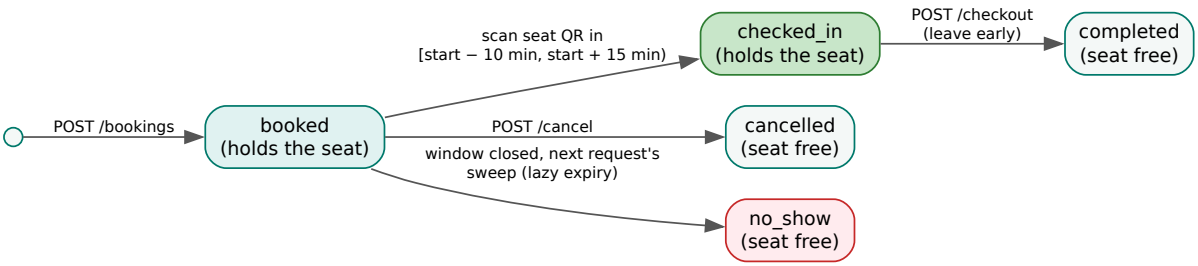


Figure 4: Booking life cycle. Only `booked` and `checked_in` hold the seat.

Lazy no-show release

The requirement “release the seat 15 minutes after the start if nobody checked in” sounds like a timer. A timer was deliberately **not** used, because the free Render server goes to sleep after 15 minutes without traffic, and a sleeping process runs no timers. Instead, the release is **lazy**:

```
UPDATE bookings SET status = 'no_show'
 WHERE status = 'booked'
   AND lower(slot) + make_interval(mins => $1) <= now()    -- $1 = 15
```

This one statement runs at the start of every request that reads or changes bookings (the `noShowSweep` middleware, and again inside the booking transaction). Whoever asks next, the expired bookings are released first, so every answer is correct even if the server slept for hours. It is cheap thanks to the partial index. The test “releases a no-show lazily” inserts a booking that started 20 minutes ago, makes one unrelated request, and checks that the row became `no_show` and the seat could be booked by someone else. The same thing is visible in the app (Fig. 12).

QR check-in

Each seat’s `qr_token` is random (`seat_` + 12 random bytes in base64url), so it cannot be guessed from the label. The student’s app sends only the scanned token to `POST /api/checkin`. The server finds **that student’s** `booked` booking **for that seat** whose window `[start − 10 min, start + 15 min)` contains now, locks it with `FOR UPDATE` (so a double tap cannot check in twice), and sets `checked_in`. If nothing matches, it works out the most useful reason, in this order: already checked in, too early (with the opening time), wrong seat (naming the right seat and room), check-in closed (it became a no-show), or no booking. An admin can issue a new token for a seat; the old printed code then stops working at once (tested).

Time rules

`api/src/lib/slots.js` checks: start and end on the 15-minute grid, length 30–240 minutes, start not before the current 15-minute block (so at 10:07 you may still book 10:00), and at most 7 days ahead. All rules come from environment variables and are served to the app by `GET /api/config`, so the app’s time pickers offer exactly what the server accepts.

API reference

All endpoints are under `/api`. Errors always look like `{"error": {"code": "seat_taken", "message": "That seat is already booked for part of this time"}}`, so the app can show the message and switch on the code.

Method	Path	Who	Does
GET	`/health`	anyone	Server and database status
GET	`/config`	anyone	Booking rules
POST	`/auth/register`	anyone	New student account → token
POST	`/auth/login`	anyone	Token (rate limited)
GET	`/auth/me`	user	Current user
GET	`/rooms`	user	Open rooms + free-now counts
GET	`/rooms/:id/availability?start&end`	user	Each seat: free / taken / mine
GET	`/rooms/:id/timeline?date`	user	Busy intervals per seat for a day
POST	`/bookings`	user	Book (transaction above)
GET	`/bookings/me?scope=upcoming|history`	user	My bookings
POST	`/bookings/:id/cancel`	owner	Cancel before check-in
POST	`/bookings/:id/checkout`	owner	Leave early
POST	`/checkin`	user	`{qrToken}` → checked-in booking
GET / POST / PATCH	`/admin/rooms`, `/admin/rooms/:id`, `/admin/rooms/:id/seats`, `/admin/seats/:id`, `/admin/seats/:id/rotate-qr`	admin	Manage rooms, seats and QR codes
GET	`/admin/bookings?date&roomId`	admin	A day’s bookings with names
GET	`/admin/stats?days`	admin	Dashboard numbers

HTTP status codes used

`200` OK, `201` created, `400` invalid input (with the first problem as the message), `401` not logged in or token expired, `403` not an admin, `404` not found (also used for another student’s booking, so the API does not reveal that it exists), `409` a rule was broken (seat taken, overlap, limit, wrong seat, too early), `429` too many login attempts, `500` unexpected error (details only in the server log), `503` database unreachable (health check).

Security

Concern	What SeatSure does
Password storage	scrypt with a random 16-byte salt per password, stored as `scrypt$salt$hash`; compared with `timingSafeEqual` so timing reveals nothing.

Login	Same message for “unknown email” and “wrong password” (no account discovery). At most 20 attempts per 15 minutes per IP.
Sessions	JWT signed with HS256, expires after 7 days, algorithm pinned on verify. The user is reloaded on every request, so a deleted account stops working at once. The app logs out automatically on any `401`.
Authorisation	Roles checked on the server (`requireAdmin`). Sign-up ignores any `role` the client sends (tested). Students only ever see their own bookings; other students’ bookings appear only as “taken”.
SQL injection	Every query uses parameters (`$1`, `$2`, …); no user text is concatenated into SQL.
Input	zod schemas on every body and query; JSON bodies limited to 20 kB.
Transport	HTTPS to Render; TLS from the API to Neon (`sslmode=require`).
Secrets	The repository is public, so no secret is in it: the database URL and JWT secret live only in Render’s settings and in a git-ignored local file. The JWT secret is generated by Render. Seeding a hosted database refuses to use the public demo passwords and never prints the real ones. A scan of every commit found no connection strings or keys.
Headers / CORS	helmet sets safe headers. CORS allows any origin, which is safe here because authentication uses a Bearer header, not cookies, so a foreign site cannot ride on a logged-in session.

The Flutter app

Structure (`app/lib`, 18 files, about 2,450 lines)

```
lib/
  main.dart                 app start, light/dark themes, login gate
  api/api_client.dart       HTTP calls, Bearer token, error -> ApiException, 60 s
timeout
  models/models.dart        User, Room, SeatAvailability, Booking, AdminStats ...
  state/session.dart        who is logged in, server address, booking rules
(ChangeNotifier)
  state/bookings_model.dart my bookings: refresh, book, cancel, check in, check
out
  screens/                  login, server settings, home shell, rooms, room (seat
map),
                            my bookings, scan (check-in), profile
  screens/admin/            dashboard, day bookings, rooms, room seats (QR sheet)
  widgets/common.dart       status chip, error/empty views, date formats
```

State management

Two `ChangeNotifiers` are provided at the top of the tree with `MultiProvider`. `Session` holds the user, token, server address and booking rules, and saves the login with `shared_preferences` so the app reopens logged in. `BookingsModel` holds the student’s bookings and is shared by the My bookings and Check in tabs, so a check-in on one tab updates the other at once. The login gate (`_Gate`) watches `Session` and swaps between the login screen and the home screen.

One codebase, three platforms

- **Responsive layout.** Below 720 px wide the home screen uses a bottom `NavigationBar`; on wider screens (web, desktop, tablets) a side `NavigationRail`. Grids use `SliverGridDelegateWithMaxCrossAxisExtent`, so they add columns as the window grows.
- **Platform differences.** Camera scanning (`mobile_scanner`) works on Android, iOS and the web, but has no Linux implementation. The Check in screen checks the platform and, on Linux (or when the camera is blocked), offers a text field for the code printed under the QR.
- **Server address.** Release builds get the API address at build time (`--dart-define=API_BASE_URL=…`). The web build uses its own origin, because the API serves it. A Server screen lets the address be changed at run time, for example to a laptop on the same hotspot if the cloud server is unreachable during a demo.
- **Live updates.** The seat map polls every 15 seconds and also refreshes after every failed booking, so a seat booked on another device turns grey without any action.

Screens

Student (Android emulator, light theme)

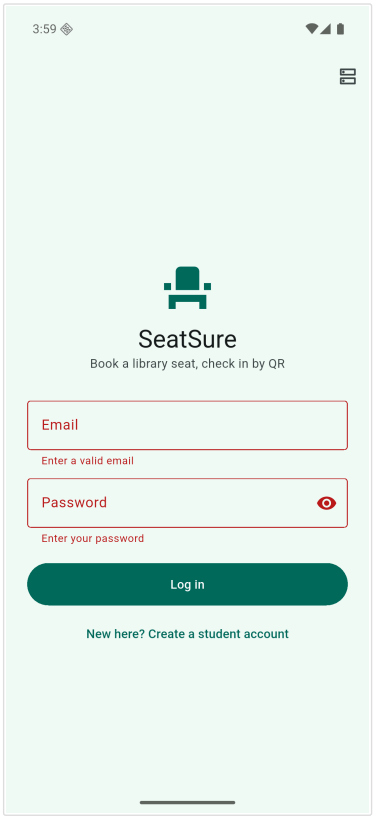


Fig. 1: Login checks input before calling the server

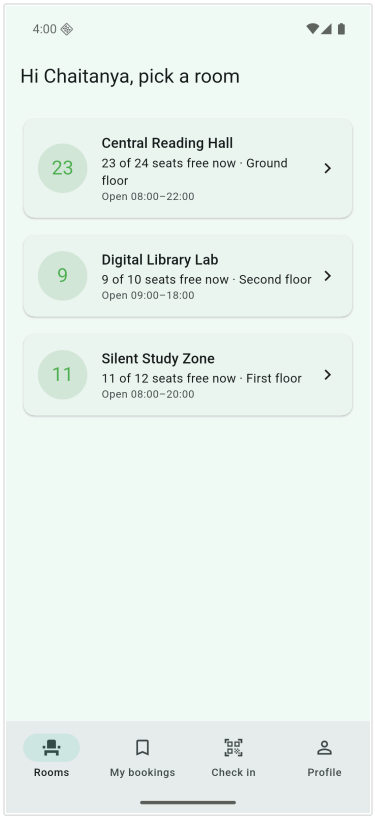


Fig. 2: Rooms with live free-seat counts

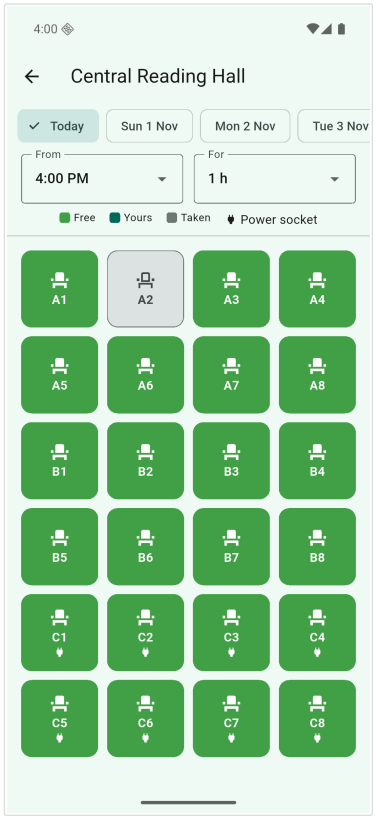


Fig. 3: Seat map for 4:00–5:00 PM; A2 taken (outlined), C row has power

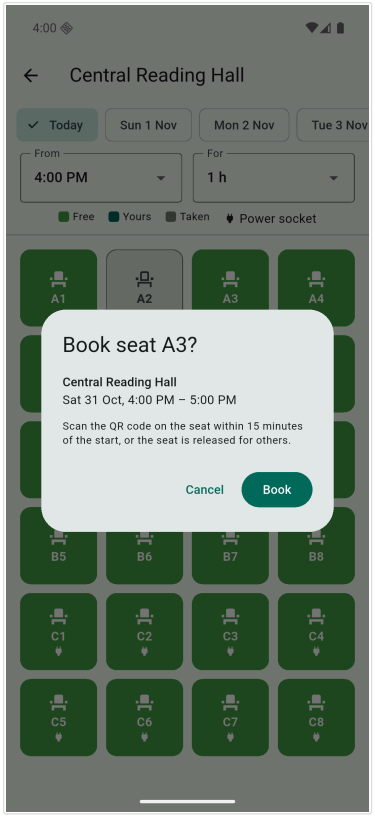


Fig. 4: Confirm dialog states the check-in rule

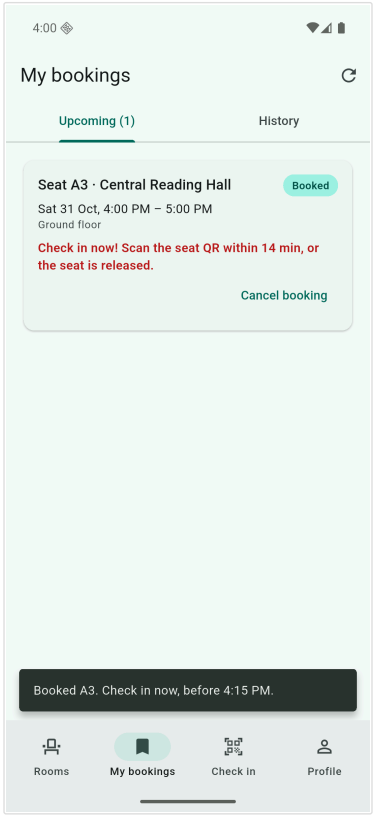


Fig. 5: Booked; the window is already open, so "Check in now"

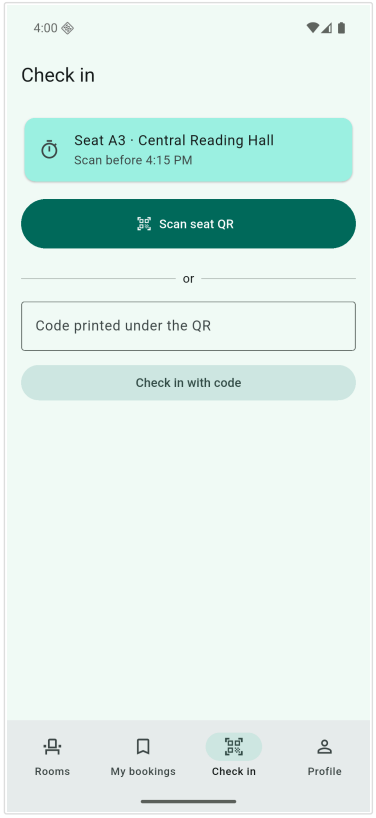
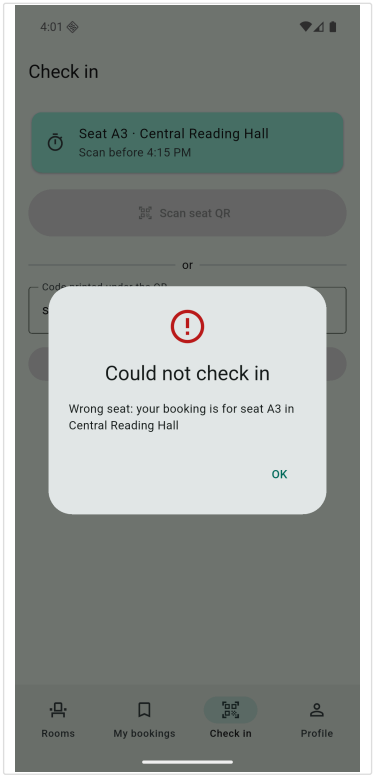
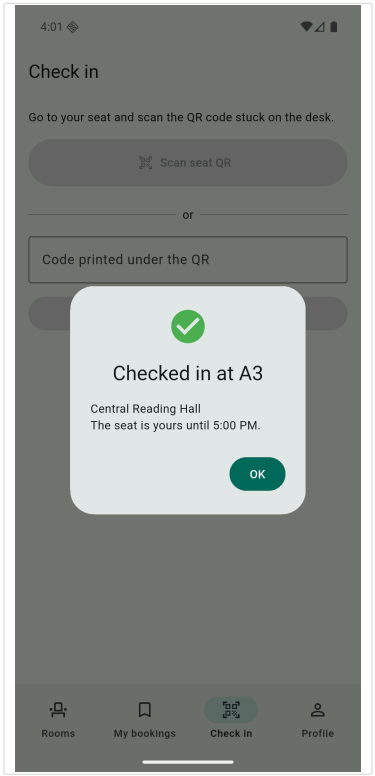


Fig. 6: Check in tab: due booking, scan button, code field





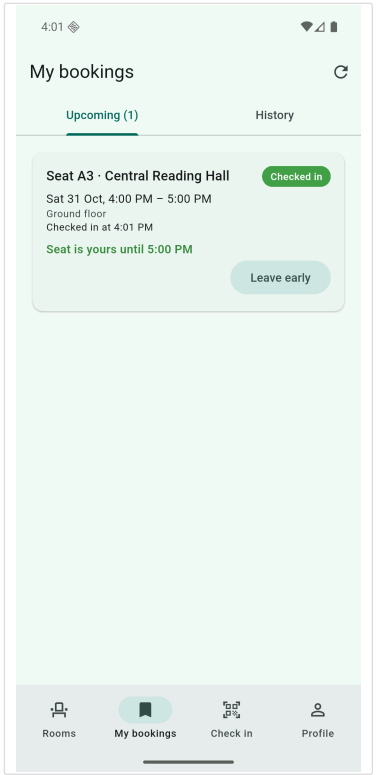
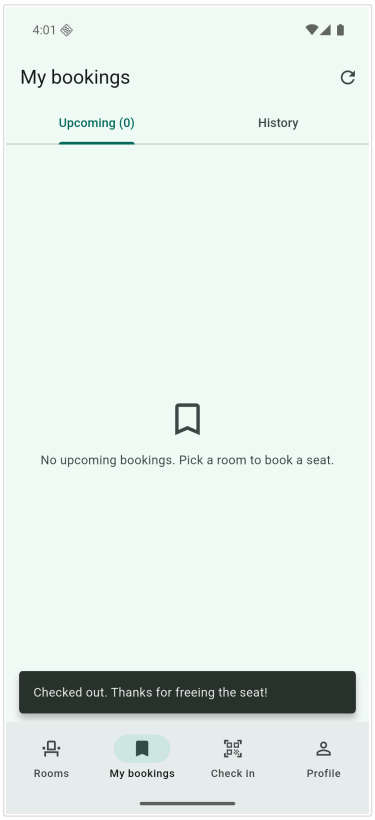
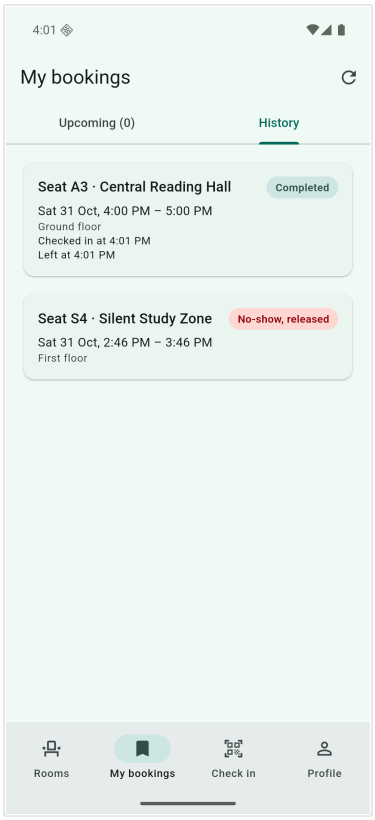


Fig. 7: A4's code scanned for an A3 booking: "Wrong seat"

Fig. 8: Right code: checked in at A3

Fig. 9: Checked in, with Leave early





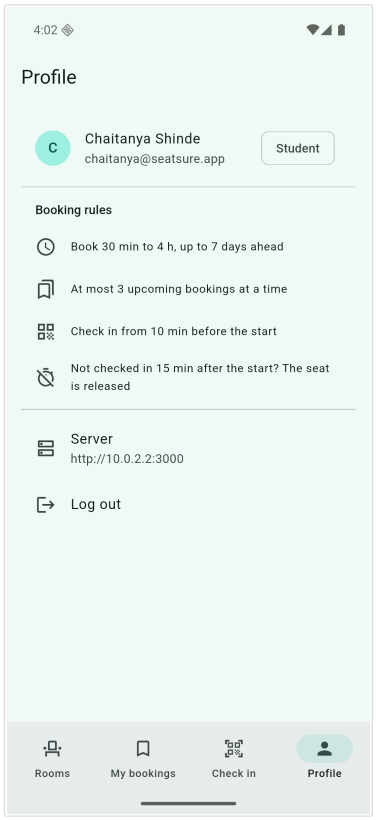


Fig. 10: Left early: seat freed at once

Fig. 11: History: completed, and an S4 booking released as a no-show by the lazy sweep

Fig. 12: Profile shows the rules served by `/api/config`

Librarian (admin)

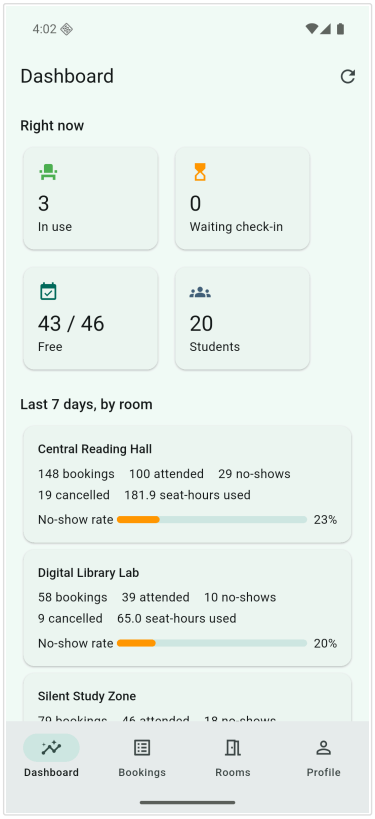


Fig. 13: Dashboard: live counts and per-room numbers

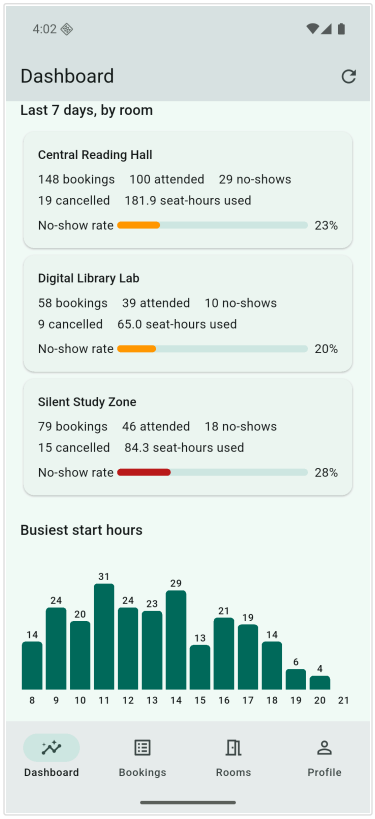


Fig. 14: No-show rates and busiest start hours (plain-widget bar chart)

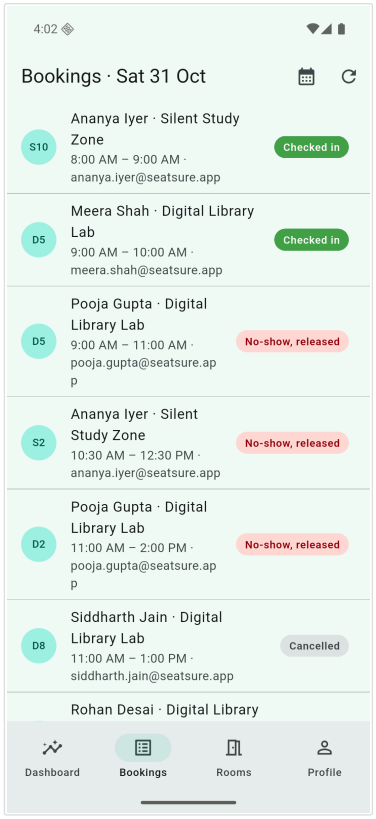


Fig. 15: Every booking of the day with names and outcomes

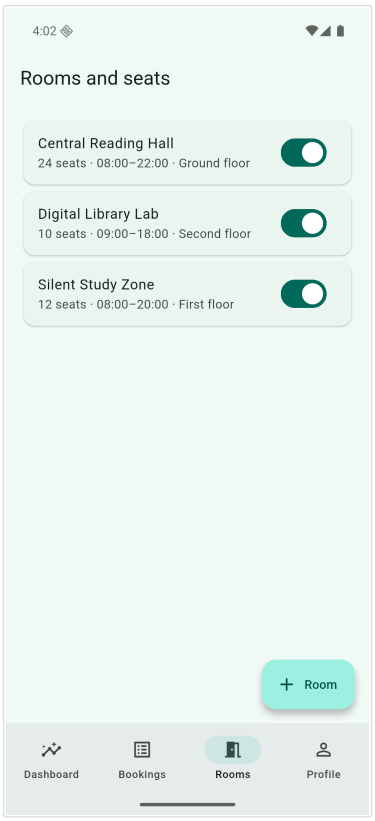


Fig. 16: Rooms: open/close switch, add room



Fig. 17: Seat QR sheet with the code under each QR

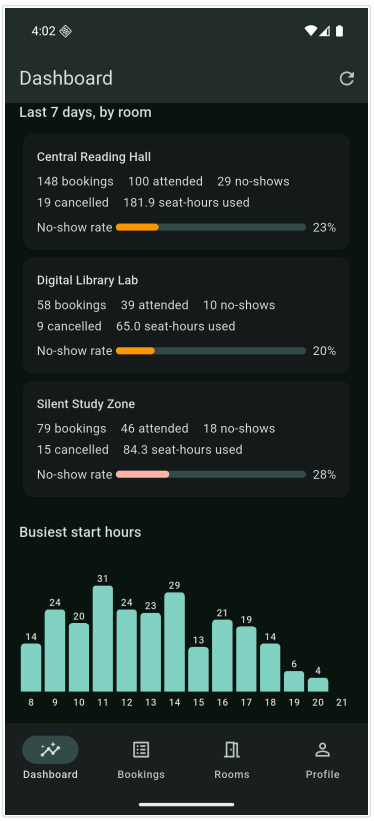


Fig. 18: Dark theme (follows the phone setting)

Dark theme and the web build

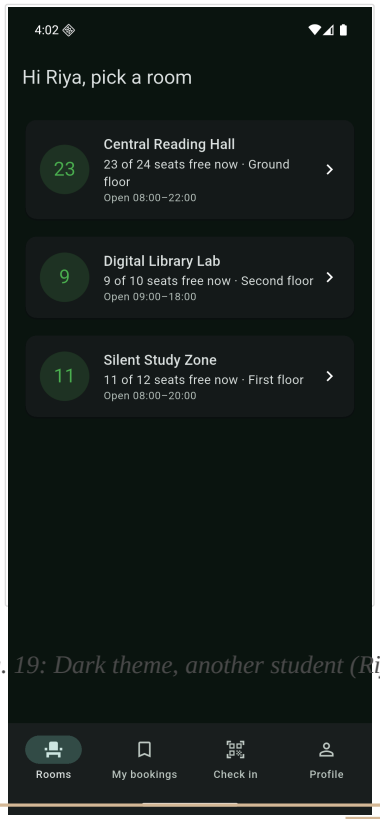


Fig. 19: Dark theme, another student (Riya)

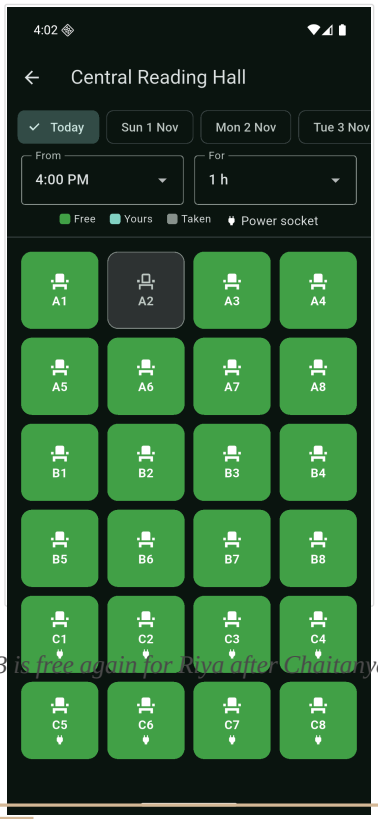


Fig. 20: A3 is free again for Riya after Chaitanya left early

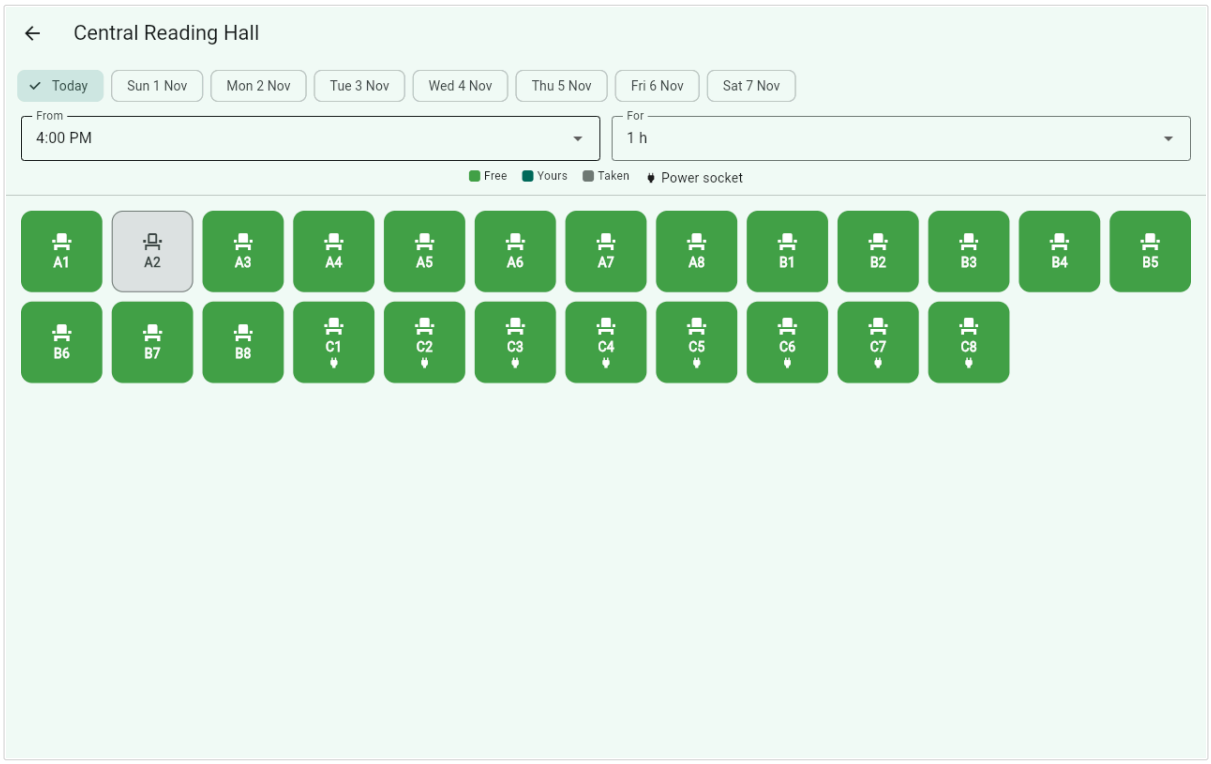


Fig. 21: Web build at 1280 px: side rail and a wider seat grid, same code

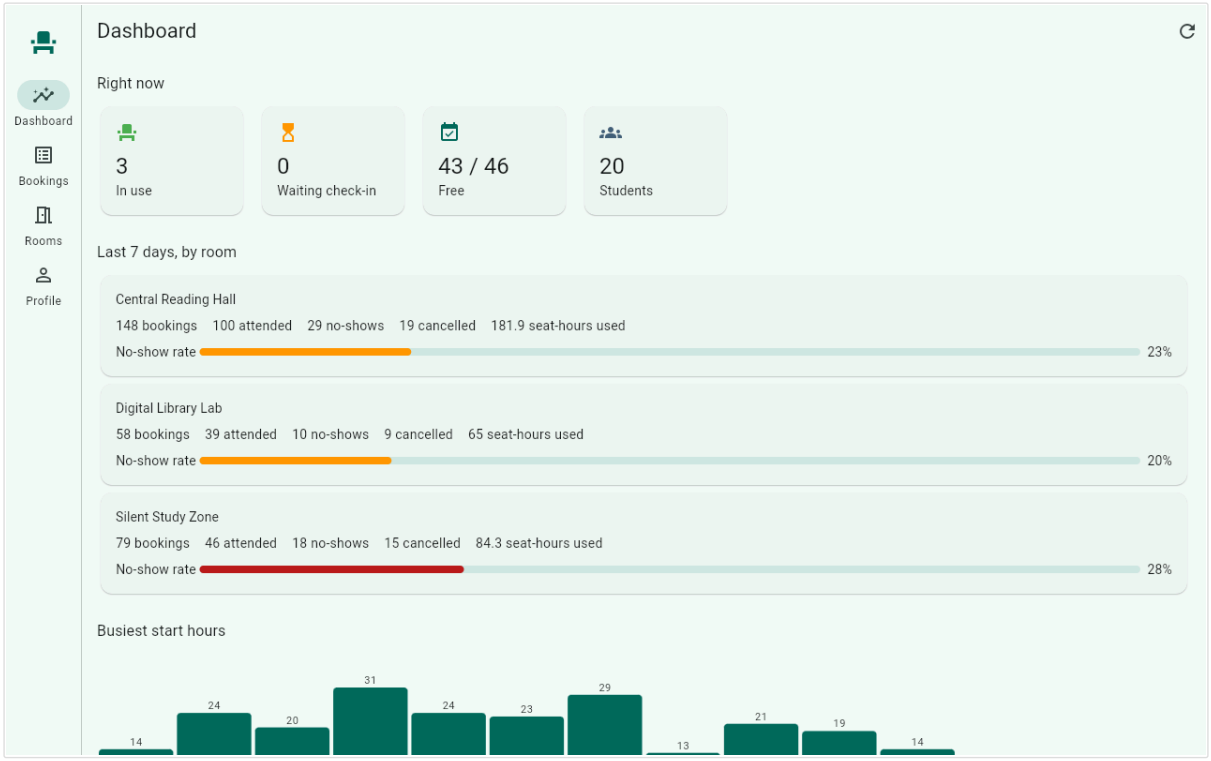


Fig. 22: Web build, admin dashboard

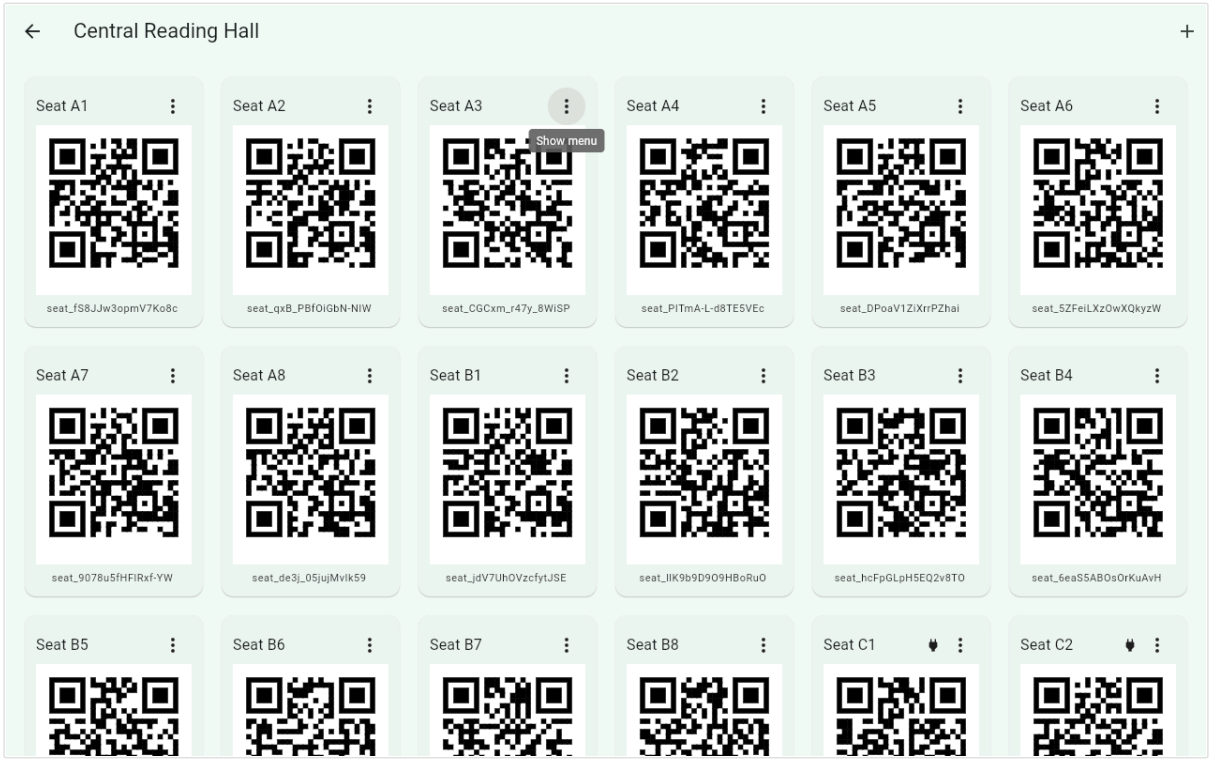


Fig. 23: Web build, printable QR sheet for a room

Testing

Automated tests

The API tests (`api/test`, `node:test` + `supertest`) run against a real PostgreSQL test database, because the most important rules live in SQL and cannot be tested with mocks. Every test file empties the database first. The Flutter tests (`app/test`) cover the models and widgets.

Suite	Tests
Auth (7)	registers a student; ignores a client-sent role; email uniqueness ignores case; readable validation errors; login works only with the right password and gives the same error for an unknown email; routes reject missing/bad tokens; health reports the database up
Booking rules (10)	books a free seat; overlapping booking of the same seat → 409; back-to-back slots allowed; one student cannot hold two seats at once; **two simultaneous requests for one seat: exactly one wins**; **per-student limit holds for parallel requests**; time rules (grid, length, past, horizon, opening hours); cancel frees the seat, cannot cancel twice or someone else's booking; seat map free/taken/mine; my bookings only
Check-in (5)	right seat inside the window; a few minutes early; too early / wrong seat / unknown code / no booking; **lazy no-show release**; leave early frees the seat
Admin (5)	closed to students; room + bulk seats, a failed batch leaves nothing behind; closing before opening rejected; rotating a QR code kills the old one; stats and the day list

Flutter (9)	JSON parsing and UTC → local time; check-in window edges; which actions each status allows; opening hours in minutes; login validation; sign-up rules; booking card texts; no-show card has no actions; **logging out does not crash** (regression test)

Checking that the tests can fail

A test that never fails proves nothing. The two concurrency tests were checked by breaking the code on purpose: without the `FOR UPDATE` lock the limit test failed 3 times out of 3, and with the `no_seat_overlap` constraint dropped the simultaneous-booking test failed. The logout regression test was also run against the old code and failed there.

Continuous integration

GitHub Actions runs three jobs on every push (`.github/workflows/ci.yml`):

1. **API tests** against a PostgreSQL 18 service container.
2. **Flutter analyze + tests** with the same Flutter version (3.47.7).
3. **Docker image**: builds exactly what Render deploys, starts it next to a fresh PostgreSQL, and checks that `/api/health` reports the database up and that the web app's page loads.

Manual test rounds

Round	Setup	Result
1. Emulator	Pixel AVD, Android 16, release APK against the API on the laptop; web build in Chromium	Every student and admin flow worked. A seat booked on Android showed as taken on the web build for another student. Found 3 bugs (below).
2. Live	Render + Neon; the student on the laptop browser (Brave, dark theme)	Booked D1, checked in at 3:54 PM inside the 3:45–4:00 window; the admin dashboard updated live. Found 1 bug (below).
3. Final	App 1.0.1 on the emulator, fresh demo data, light and dark themes	All flows in Figs. 1–20 including wrong seat, leave early and a lazy no-show.

Bugs found by testing, and fixes

Bug	Cause	Fix
Grey screen after logging out (release build)	Screens still on display rebuilt once with no user and hit `user!` (null check).	Null-safe reads during logout; regression test.
Debug-mode error on reaching the home screen	The home screen notified listeners during the build phase.	Start-up work moved to a post-frame callback.
Web page fallback crashed the server	`res.sendFile` needs an absolute path; `WEB_DIR` was relative.	`path.resolve` on `WEB_DIR`.
Code field invisible on a wide browser	The square camera box filled a 1920 px wide window (about 1700 px tall).	Check-in column limited to 480 px; clearer "camera blocked" hint (app 1.0.1).

First Render deploy failed	Running as root, `tar` tried to restore file owners, which Render's builder forbids.	Flutter built as a normal user inside Docker.

Deployment

Piece	How
Docker image	Two stages. Stage 1 (Debian) installs Flutter 3.47.7 as a non-root user and runs `flutter build web`. Stage 2 (`node:24-alpine`) installs only production dependencies, copies the API and the built web files, runs as the `node` user, applies migrations and starts the server. The prebuilt Flutter images were not used because they lag behind and failed the app's Dart version constraint.
Render	`render.yaml` Blueprint: free Docker web service in Singapore, health check on `/api/health`, auto-deploy on every push to `main`. `DATABASE_URL` is entered in the dashboard; `JWT_SECRET` is generated by Render.
Neon	Free PostgreSQL 18 project in Singapore (the same region as the API, because each request makes several queries). Seeded once from the laptop with private passwords.
Android	`flutter build apk --release --split-per-abi --dart-define=API_BASE_URL=https://seatsure-9hn0.onrender.com` gives a 26 MB arm64 APK.

Free-tier behaviour

Render's free service sleeps after 15 minutes without requests; the next request waits 30–60 seconds while it wakes. The app allows up to 60 seconds per request for this reason, and the demo starts by opening `/api/health` a couple of minutes early. Lazy no-show release (Section 6.3) is what keeps the booking rules correct across these sleeps.

Limitations and future scope

- **Polling, not push.** The seat map refreshes every 15 seconds. WebSockets or Server-Sent Events would show changes instantly.
- **No push notifications.** The reminder to check in is an in-app countdown. Firebase Cloud Messaging could notify the phone before the window closes.
- **Static QR codes.** A photo of a seat's QR code could be shared to check in remotely. The admin can replace a code at once; a stronger design would show a rotating code on a small screen at each desk.
- **No penalties.** Repeated no-shows are counted but not punished; a temporary booking ban would be a small addition.
- **Accounts.** No email verification, password reset or college single sign-on.
- **Time zone.** Opening hours are in one configured zone (Asia/Kolkata); a multi-campus version would store a zone per room.
- **iOS / macOS** builds need a Mac with Xcode; the code itself has nothing Android-specific apart from the manifest.

Conclusion

SeatSure meets the ESE requirements: it is built with one CPAD framework (Flutter), runs on Android, the web and Linux desktop from a single codebase, and has an integrated backend (Express REST API) and database (PostgreSQL). Beyond the minimum, the two hard problems of seat booking are solved at the right layer: double booking is impossible because PostgreSQL refuses overlapping ranges, and no-shows are released correctly even on a server that sleeps, because expiry is computed when data is read instead of by a timer. 36 automated tests, a CI pipeline that also builds the deployed image, and three manual test rounds back these claims, and the system is live on the internet.

Appendix A: Running the project

```
# database (once)
psql -h 127.0.0.1 -U postgres -c "CREATE ROLE seatsure LOGIN PASSWORD
'seatsure_local'"
psql -h 127.0.0.1 -U postgres -c "CREATE DATABASE seatsure_dev OWNER seatsure"

# API
cd api && cp .env.example .env && npm install
npm run seed        # migrations + demo data
npm run dev         # http://localhost:3000/api/health
npm test            # 27 API tests (needs seatsure_test)

# app
cd app && flutter test
flutter run -d linux              # desktop -> localhost:3000
flutter run -d emulator-5554      # Android emulator -> 10.0.2.2:3000
```

Local demo logins (public, only valid on a local database): `admin@seatsure.app` / `Admin@1234`, `chaitanya@seatsure.app` and `riya@seatsure.app` / `Student@123`. The live deployment uses private passwords.

Appendix B: Where is X

Thing	File
Tables and constraints	`api/migrations/001_init.sql`
Booking transaction	`api/src/routes/bookings.js`
QR check-in and its messages	`api/src/routes/checkin.js`
Lazy no-show release	`api/src/middleware/noShowSweep.js`
Time rules	`api/src/lib/slots.js`
Password hashing / tokens	`api/src/lib/password.js`, `api/src/lib/tokens.js`
DB error → 409 message	`api/src/middleware/errorHandler.js`
Admin routes and statistics SQL	`api/src/routes/admin.js`
App HTTP client	`app/lib/api/api_client.dart`
Login state	`app/lib/state/session.dart`
Seat map and booking	`app/lib/screens/room_screen.dart`
QR scanning	`app/lib/screens/scan_screen.dart`

Tests	`api/test/`, `app/test/`
Deployment	`Dockerfile`, `render.yaml`, `.github/workflows/ci.yml`

END OF REPORT

CHAITANYA SHINDE | Final Year BTECH CE 231070066